IEEE 829 Test Case Specification

IBA Campus Facility Booking System — QA Artifact

Prepared by:

Abdul Wasay Imran (27126)   Saad Imam (27079)

Muhammad Imad Raza (26953)   Syed Muhammad Deebaj Haider Kazmi (26012)

Version 1.0 — May 28, 2026

Contents

1 Introduction

1.1 Purpose

This document defines the formal Test Case Specification for the IBA Campus Facility Booking System (CFBS). It details the inputs, procedural execution steps, expected outcomes, and actual execution statuses observed during the automated and manual test runs.

1.2 Scope Traceability

These test cases are designed to validate all functional requirements documented in the CFBS Software Requirements Specification (SRS). They span multiple test levels:

- **Unit Level (NestJS / Jest):** Validating individual services, helper utilities, and controllers in isolation.
- **Integration Level (Supertest / Express):** Testing REST API endpoint routing, middleware authorization, and database persistence.
- **E2E System Level (Playwright / Chromium & Firefox):** Emulating complete user workflows in headless and headed browser contexts.

1.3 Environmental Procedural Requirements

To execute the test cases specified herein, the following sandbox parameters are strictly required:

1. A dedicated, isolated testing database seeded via `seedTestData.sql` before each test suite execution.
2. A local test server listening on port `3000` (Backend API) and a Vite development server on port `5173` (Frontend Client).
3. Automation frameworks configured using Node 18+, Jest v30, and Playwright v1.60.

2 Test Case Specifications

2.1 Authentication Authorization (TC-AUTH)

TC-AUTH-001: Valid Student Login Validation

Test Level: E2E & Integration
Type: Positive / Functional
Traceability (SRS): REQ-1, REQ-3, REQ-4
Status: PASS

Preconditions: Database is seeded with user `test-student` and password `testpass`.
Procedural Steps:

1. Send a POST request to `/api/auth/login` with student credentials or enter them on the login page.
2. Verify that the response returns an `access_token` and a payload containing the student's profile.
3. Confirm that the student is redirected to `/dashboard` and that the plain text password is not returned.

TC-AUTH-002: Valid Program Office Login Validation

Test Level: E2E & Integration
Type: Positive / Functional
Traceability (SRS): REQ-2, REQ-3, REQ-4
Status: PASS

Preconditions: Database is seeded with PO user `test-po` and password `testpass`.
Procedural Steps:

1. Submit a POST request to `/api/auth/login` with PO credentials.
2. Assert that a valid JWT token is returned and that the role is identified as `programoffice`.
3. Verify that the user is redirected to the Program Office dashboard.

TC-AUTH-003: Invalid Credentials Handling

Test Level: E2E, Integration, Unit
Type: Negative / Security
Traceability (SRS): REQ-5
Status: PASS

Preconditions: The auth database is active and populated.
Procedural Steps:

1. Send a login request using an invalid ERP or password (e.g., username: `wrong-user`, password: `wrong-pass`).
2. Assert that the API rejects the request with a `401 Unauthorized` status.
3. Verify that the UI displays the error message: "Invalid credentials".

TC-AUTH-004: Protected Route Access Redirection

Test Level: Integration (RBAC)
Type: Security / Negative
Traceability (SRS): REQ-6
Status: PASS

Preconditions: The user is unauthenticated (no session token stored).
Procedural Steps:

1. Attempt to directly access a protected URL (e.g., `GET /api/bookings`).
2. Verify that the API rejects the request with a `401 Unauthorized` status.

3. Confirm that the UI redirects the unauthenticated user back to the login page.

2.2 Room Booking Request (TC-BOOK)

TC-BOOK-001: Happy Path Booking Creation

Test Level: E2E & Integration
Type: Positive / Functional
Traceability (SRS): REQ-1, REQ-2, REQ-3, REQ-6
Status: PASS (Firefox Cleanup Warning)

Preconditions: Database is seeded; user `test-student` is authenticated.
Procedural Steps:

1. Select an active building (Test Building) and room (Test Room A) from the dropdown options.
2. Choose a date in the current year and select an available time slot (Slot 3: 11:30 AM).
3. Enter a valid booking purpose and submit the request.
4. Assert that the request is created with a status of `pending` and appears in the student's dashboard.

TC-BOOK-002: Overlapping Booking Prevention

Test Level: E2E & Integration
Type: Negative / Constraint
Traceability (SRS): REQ-4, REQ-7
Status: PASS

Preconditions: A pending booking exists for Room A on a specific date and time slot.
Procedural Steps:

1. Authenticate as a student.
2. Attempt to submit a new booking request for the exact same room, date, and slot.
3. Verify that the system rejects the request with a `409 Conflict` status and does not submit it.

TC-BOOK-003: Blocked Slot Booking Prevention

Test Level: E2E & Integration
Type: Negative / Constraint
Traceability (SRS): REQ-4, REQ-7
Status: PASS

Preconditions: An Admin has blocked Slot 2 on the target date.
Procedural Steps:

1. Authenticate as a student.
2. Attempt to book Slot 2 on the blocked date for the specified room.
3. Assert that the system rejects the request with a `409 Conflict` status, returning an error: "This slot is blocked by admin".

TC-BOOK-004/5/6/7: Booking Boundary Verification

Test Level: Integration
Traceability (SRS): REQ-3
Type: Boundary
Status: FAIL (DEF-010)

Preconditions: The database is active; student is authenticated.
Procedural Steps:

1. Send a POST request to `/api/bookings` with an invalid `slot_id` (e.g., lower bound: `0`, upper bound: `8`).
2. Assert that the API rejects the request with a `400 Bad Request` validation error.
3. **Actual Result:** The system throws an unhandled database foreign key violation, returning a `500 Internal Server Error` instead.

TC-BOOK-008: Missing Purpose Field Validation

Test Level: E2E & Integration
Traceability (SRS): REQ-5
Type: Negative / Validation
Status: PASS

Preconditions: The student is logged in and preparing a booking form.
Procedural Steps:

1. Select a building, room, date, and slot, but leave the booking "purpose" field empty.
2. Click "Submit Booking Request."
3. Verify that native HTML5 validations or custom UI alert boxes block the submission and highlight the missing field.

TC-BOOK-009: Simultaneous Booking Concurrency

Test Level: Integration
Traceability (SRS): REQ-7
Type: Concurrency / Stress
Status: FAIL (DEF-002)

Preconditions: Two students are logged in simultaneously.
Procedural Steps:

1. Fire two booking requests for the exact same room, date, and slot at the exact same millisecond.
2. Assert that one request succeeds (`201 Created`) while the other is gracefully rejected with a `409 Conflict`.
3. **Actual Result:** The database unique constraint throws an unhandled exception on the duplicate insert, returning a `500 Internal Server Error` instead of a `409`.

2.3 Booking Management — Program Office (TC-PO)

TC-PO-001: View Pending Requests Dashboard

Test Level: E2E & Integration
Type: Positive / Functional
Traceability (SRS): REQ-1
Status: PASS

Preconditions: A student has submitted a pending booking request.
Procedural Steps:
1. Log in to the portal using Program Office credentials (`test-po`).
2. Access the pending requests tab.
3. Assert that the queue correctly displays the student's name, room, date, slot, and purpose.

TC-PO-002: Approve Pending Request

Test Level: E2E & Integration
Type: Positive / Functional
Traceability (SRS): REQ-2, REQ-3, REQ-4
Status: PASS

Preconditions: A pending request exists in the Program Office queue.
Procedural Steps:
1. Log in as a PO member and select the target pending request.
2. Click "Approve."
3. Verify that the booking status is updated to `approved` and moves to the approved bookings tab.

TC-PO-003: Reject Pending Request

Test Level: E2E & Integration
Type: Positive / Functional
Traceability (SRS): REQ-2, REQ-4
Status: PASS

Preconditions: A pending request exists in the Program Office queue.
Procedural Steps:
1. Log in as a PO member and select the target pending request.
2. Click "Reject."
3. Assert that the booking status changes to `rejected` and the time slot remains available for other users.

TC-PO-004: Auto-Rejection of Overlapping Bookings

Test Level: E2E & Integration
Type: Functional / Logic
Traceability (SRS): REQ-5
Status: FAIL (DEF-002)

Preconditions: Two pending booking requests exist for the same room and slot.
Procedural Steps:
1. Log in as a PO member.
2. Approve the first student's pending request.
3. Verify that the second student's request is automatically updated to `rejected` or flagged as a conflict.
4. **Actual Result:** The test cannot proceed past the setup phase because the database unique constraint blocks the submission of the second overlapping request, preventing it from ever appearing in the PO's queue.

2.4 Booking Cancellation (TC-CANCEL)

TC-CANCEL-001: Student Cancels Own Pending Booking

Test Level: E2E & Integration
Type: Positive / Functional
Traceability (SRS): REQ-1, REQ-2, REQ-3
Status: FAIL (DEF-001)

Preconditions: A student has a booking request with a `pending` status.
Procedural Steps:
1. Log in as the student who created the booking and locate it on the dashboard.
2. Click “Cancel” and accept the confirmation dialog.
3. Assert that the booking status changes to `cancelled`.
4. **Actual Result:** The booking status is incorrectly updated to `rejected` in the database.

TC-CANCEL-002: Student Cancels Own Approved Booking

Test Level: E2E
Type: Positive / Functional
Traceability (SRS): REQ-1, REQ-2, REQ-3
Status: FAIL (DEF-004)

Preconditions: A student has a booking request with an `approved` status.
Procedural Steps:
1. Log in as the student who created the booking and view the dashboard.
2. Locate the card for the approved booking.
3. Click “Cancel” and accept the confirmation dialog.
4. **Actual Result:** The “Cancel” button is completely missing from the UI card on the student dashboard for approved bookings.

TC-CANCEL-005: PO Cancels Approved Booking

Test Level: E2E & Integration
Type: Positive / Functional
Traceability (SRS): REQ-1, REQ-2, REQ-3
Status: FAIL (DEF-001)

Preconditions: An approved booking exists in the system.
Procedural Steps:
1. Log in as a PO member and access the approved bookings tab.
2. Locate the target booking, click “Cancel,” and accept the confirmation dialog.
3. Assert that the booking status changes to `cancelled`.
4. **Actual Result:** While the UI displays a cancellation success alert, the database incorrectly records the final status as `rejected`.

2.5 Admin System Management Data Integrity (TC-ADMIN / TC-DATA)

TC-ADMIN-001: Create Student Account

Test Level: E2E & Integration
Type: Positive / Functional
Traceability (SRS): REQ-1, REQ-3
Status: PASS

Preconditions: The Admin is logged in.
Procedural Steps:

1. Access the student management tab.
2. Enter a unique ERP, name, email, and password, and submit the form.
3. Verify that the student record is created in the database and visible in the UI.

TC-ADMIN-002: Create Duplicate Student Account

Test Level: E2E & Integration
Type: Negative / Functional
Traceability (SRS): REQ-2, REQ-4
Status: PASS

Preconditions: A student record with ERP `12345` already exists.
Procedural Steps:

1. Attempt to create a new student account using the existing ERP `12345`.
2. Assert that the system rejects the creation with a `409 Conflict` status and displays a unique constraint error.

TC-ADMIN-006/007: Block Unblock Slot Lifecycle

Test Level: Integration
Type: Positive / Functional
Traceability (SRS): REQ-1, REQ-2, REQ-3
Status: PASS

Preconditions: The Admin is logged in.
Procedural Steps:

1. Send a POST request to `/api/blocked-slots` to block Slot 1 on a specific room and date.
2. Verify that the block is recorded in the database.
3. Send a DELETE request to `/api/blocked-slots/:id` to unblock the slot.
4. Assert that the system returns a `200 OK` status with the message: “Slot unblocked”.

TC-DATA-001: Audit Trail Verification

Test Level: Integration
Type: Functional / Security
Traceability (SRS): REQ-8
Status: FAIL (DEF-015)

Preconditions: A booking has been approved by the Program Office.
Procedural Steps:

1. Retrieve the approved booking record by sending a GET request to `/api/bookings/:id`.
2. Assert that the returned payload contains the `reviewed_by` field containing the PO member’s UUID.
3. **Actual Result:** The `reviewed_by` field is explicitly omitted from the returned database payload, breaking the audit trail.

---

3 Test Run Execution Summary

3.1 Execution Dashboard

The following table summarizes the execution results of the automated Jest and Playwright test runs, mapping failures directly to the logged defects.

Table 1: Overall Test Case Execution Log

Test ID	Suite File	Level	Executed	Observed Behavior	Status
TC-AUTH-001	`auth`	E2E	Yes	User logged in, token returned	**PASS**
TC-AUTH-002	`auth`	E2E	Yes	PO logged in, dashboard accessible	**PASS**
TC-AUTH-003	`auth`	E2E	Yes	Handled bad credentials gracefully	**PASS**
TC-AUTH-004	`auth.rbac`	Int	Yes	Unauthenticated request blocked	**PASS**
TC-AUTH-007	`auth`	E2E	Yes	User logged out successfully	**PASS**
TC-BOOK-001	`student-booking`	E2E	Yes	Booking created as pending	**PASS***
TC-BOOK-002	`student-booking`	E2E	Yes	Overlap rejected with 409	**PASS**
TC-BOOK-003	`bookings.create`	Int	Yes	Rejected booking on blocked slot	**PASS**
TC-BOOK-004	`bookings.create`	Int	Yes	Threw 500 on lower boundary limit	**FAIL** (DEF-010)
TC-BOOK-007	`bookings.create`	Int	Yes	Threw 500 on upper boundary limit	**FAIL** (DEF-010)
TC-BOOK-008	`student-booking`	E2E	Yes	Prevented empty purpose	**PASS**
TC-BOOK-009	`bookings.concurrency`	Int	Yes	Threw 500 on concurrent race	**FAIL** (DEF-002)
TC-PO-001	`po-approval`	E2E	Yes	Dashboard populated with pending	**PASS**
TC-PO-002	`po-approval`	E2E	Yes	Status updated to approved	**PASS**
TC-PO-003	`po-approval`	E2E	Yes	Status updated to rejected	**PASS**
TC-PO-004	`po-approval`	E2E	Yes	Overlap blocked during setup	**FAIL** (DEF-002)
TC-CANCEL-001	`cancellation`	E2E	Yes	Status updated to rejected	**FAIL** (DEF-001)
TC-CANCEL-002	`cancellation`	E2E	Yes	Cancel button missing from DOM	**FAIL** (DEF-004)
TC-CANCEL-005	`cancellation`	E2E	Yes	Status updated to rejected	**FAIL** (DEF-001)
TC-ADMIN-001	`admin`	E2E	Yes	Student successfully added	**PASS**
TC-ADMIN-002	`admin`	E2E	Yes	Duplicate ERP registration blocked	**PASS**

Test ID	Suite File	Level	Executed	Observed Behavior	Status
TC-ADMIN-003	`admin`	E2E	Yes	PO member added successfully	**PASS**
TC-ADMIN-004	`admin`	E2E	Yes	Building added successfully	**PASS**
TC-ADMIN-005	`admin`	E2E	Yes	Room added successfully	**PASS**
TC-ADMIN-006	`admin.rooms`	Int	Yes	Slot blocked by Admin successfully	**PASS**
TC-ADMIN-007	`admin.rooms`	Int	Yes	Slot unblocked by Admin successfully	**PASS**
TC-DATA-001	`bookings.data-integrity`	Int	Yes	reviewed_by omitted in payload	**FAIL** (DEF-015)

Note: Firefox E2E environment warning encountered on browser context cleanup; functional logic passed. Also, .spec.ts and .test.ts was removed from suite file names to fit table

3.2 Metrics Analysis

- **Total Test Cases Specifed:** 27
- **Total Test Cases Executed:** 27
- **Total Test Case Passes:** 18
- **Total Test Case Failures:** 9
- **Core Failure Areas:** Core failures are concentrated in the booking boundary validation layer, the booking cancellation state-machine logic, and the database unique constraint configuration on inactive bookings.